Informe de Investigación del Proceso de Desarrollo

Construcción de API REST, manejo de errores y pruebas HTTP en NestJS (nest-proyecto)

El presente documento sintetiza de forma cronológica y estructurada el proceso de investigación y guía interactiva seguido durante el diseño e implementación del proyecto backend **nest-proyecto** desarrollado en NestJS. A continuación, se detallan las consultas realizadas, sus objetivos técnicos, las palabras clave e información obtenida, así como la progresión lógica entre cada fase del proyecto.

Registro de Consultas y Flujo Metodológico

#	Consulta / Prompt	Objetivo de la consulta	Información o palabras clave obtenidas	Progresión hacia la siguiente consulta
1	Hola, quiero crear una API REST en NestJS para gestionar un recurso de usuarios (users)...	Definir la arquitectura inicial del módulo users en la API REST.	NestJS, src/users, dto, interfaces, create-user.dto.ts, update-user.dto.ts, user.interface.ts	Una vez establecida la estructura de archivos, se procedió a solicitar la lógica de negocio para gestionar los datos.
2	Ya tengo la estructura básica. Ahora quiero implementar el servicio UsersService...	Implementar las operaciones principales en el servicio gestionando excepciones.	@Injectable(), UsersService, users: User[], findAll, findOne, NotFoundException	Con la lógica del servicio en marcha, surgió el requerimiento docente de ampliar las capacidades del controlador.
3	Mi profesor me dijo que tambien debo agregar HEAD y OPTIONS me enseñas a hacerlo?...	Incorporar los métodos HTTP HEAD y OPTIONS en el controlador evitando romper el código existente.	@Head(), @Options(), UsersController, users.controller.ts, @Res(), setHeader('Allow', ...)	Al aplicar el código en el controlador, la compilación falló y se requirió consultar sobre el error en la terminal.
4	Agregué el código como me dijiste, pero al guardar en VS Code me salió un error en la terminal...	Diagnosticar y resolver el fallo de importación/tipado en TypeScript.	src/users/users.controller.ts:15:10, Response was imported here, @types/express, express	Tras solucionar el error de TypeScript, se verificó en la consola el estado del servidor y el mapeo de rutas.
5	¡Listo! Ya se corrigió el error de TypeScript y en la terminal me aparecieron estos mensajes en verde...	Validar el arranque del servidor y el correcto registro de las rutas mapeadas.	RouterExplorer, Mapped {/users, POST}, PUT, DELETE, HEAD, OPTIONS, NestApplication successfully started	Confirmadas las rutas en consola, se pasó a realizar las pruebas de integración en el cliente de HTTP.

#	Consulta / Prompt	Objetivo de la consulta	Información o palabras clave obtenidas	Progresión hacia la siguiente consulta
6	Abrí la extensión Thunder Client en VS Code para probar la petición OPTIONS...	Probar el endpoint OPTIONS /users y verificar las cabeceras de respuesta obtenidas.	Thunder Client, OPTIONS http://localhost:3000/users, 204 No Content, allow: GET, POST, HEAD, OPTIONS, x-powered-by: Express	Proceso completado exitosamente con la validación total de la API.

Conclusión del Proceso

A través de este flujo iterativo de trabajo, se logró la construcción estructurada del módulo de usuarios, la resolución efectiva de conflictos de tipos en TypeScript/Express, y la correcta implementación y verificación de los métodos HTTP avanzados requeridos por la cátedra.